



Lebanese University
Faculty of Sciences
Computer science
Licence

I2206

Data Structures

Semester: 2

Academic Year: 2023/2024

Credits: 5

Dr Zein Al Abidin Ibrahim

Hash

What is hashing?

Why hashing?

- Hashing is a technique used to store and retrieve data as quickly as possible.
- They are used to perform an optimal search.
- In trees, we have seen that balanced binary search trees support operations like insertion, deletion, and search in $O(\log n)$.
- Hashing provides a way if we need these operations in $O(1)$.

Motivation: Example

- Give an algorithm for printing the first repeated character in a character array of size n .
- The simple solution: for each character, check whether that character is repeated or not.
 - The time complexity is $O(n^2)$.
- Best solution: What happens if we save the already seen characters in an array?
 - We know that the number of possible characters is 256 (only ASCII ones).
 - Create an array of size 256 and initialize it with all zeros. For each character entered, navigate to the corresponding position and increment its value.
 - While reading the input, if we get a character whose counter is already 1, then we can say that the character is the one repeating the first time.

Motivation: Example

```
int FirstRepeatedChar(char *str, char *c) {
    int i, L;
    int count[256];

    for(i=0;i<256;i++){
        count[i]=0;
        for(i=0; str[i]!='\0';i++){
            if(count[str[i]]==1){
                *c=str[i];
                return 1;
            }
            else
                count[str[i]]++;
        }
    }
    return 0;
}
```

Motivation: Example

- In the previous example, we used an array of size 256 because we know in advance the number of different possible characters.
- Suppose the given array has numbers instead of characters, how to solve the problem?
- In this case, the set of possible values is infinite (or very large).
- It is virtually impossible to create a large table and store the counters.
- This means that there is a set of keys (universes) and a limited number of memory locations.
- **If we want to solve the problem, we need to map all these possible keys to possible memory locations.**

Hash Components

➤ 4 key components:

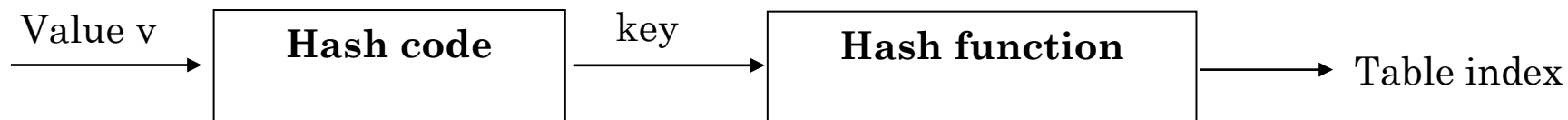
1. Hash table
2. Hash functions (hash code, key, index)
3. Collisions
4. Collision resolution techniques

Hash table

- The hash table is a generalization of an array.
- With an array, we store element e whose key is k at position i in the array.
- To search for an element e with a key k , we look at the i -th table position. This is called the **direct addressing**.
- Direct addressing is applicable when we can allocate an array with a position for each possible key. If we have fewer locations and more possible keys, simply implementing an array is not enough.
- Solution: Using another structure → hash table
The hash table or hash map (associative table, etc.) is a data structure that stores keys and their associated values. The hash table uses a hash function to map keys to their associated values.

Code and Hash Function

- Hash code or key is an integer code generated from the element contents.
 - Best if two different items have two different codes
- The hash function must calculate for a key with a value v (key) the index of the table box or put the value.
- The best thing is that this function produces two different indices for two different values.
- In practice this is impossible.



Example

➤ `char *Tab[13]`

➤ `Strings: {Khodor, Yunes, Odile}`

➤ Hash function `h`:

- Associate a number between 1 and 26 with each letter.

a	b	c	d	e	f	g	h	i	j	k	L	m	n	o	p	q	r	s	t	u	v	w	x	y	z
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

- $h(\text{word}) = (\text{sum}(\text{letters}(\text{word})) + \text{nb_letters}(\text{word})) \% 13$

➤ Example: $h(\text{Khodor}) = (71+6)\%13 = 12$

$h(\text{Yunes}) = (84+5)\%13 = 11$

$h(\text{Odile}) = (45+5)\%13 = 11$

Examples of hash codes

- Let value = word (character string)
- Each letter is coded on 5 bits (A = 00001, B = 00010,...)
- The hash code of a word is obtained by concatenating the codes of its letters (BAC = 00010 00001 00011) = 2083

Hash functions

- Two types of functions:
 - a. Depends on data distribution (e.g. student IDs)
 - b. Does not depend on distribution
 - 1. Division method
 - 2. Folding or compression method
 - 3. Mean square method
 - 4. Extraction method
 - 5. Multiplication method

Division

➤ $h(E) = E \% m$

Discussion:

➤ Depend on m

➤ If m is even,

- Even keys $\rightarrow$ even positions,
- Even keys $\rightarrow$ even positions
 $\Rightarrow$ not uniformly distributed
- In general, choosing a prime number for m gives good results

➤ Example

Let $m = 37$

YET = 11001 00101 10100 $\Rightarrow h(\text{YET}) = 25780 \% 37 = 28$

YOU = 1100101111 10101 $\Rightarrow h(\text{YOU}) = 26101 \% 37 = 16$

Folding or Compression

- Divide the word into substrings of equal size, then:
 - Add the parts without restraint
 - Reverse even parts then add without carry
 - Apply XOR between these parts.
 - ...

Example

• Key	3205	7148	2345
• Parts	32 05	71 48	23 45
• H(x)	37	19	68
Option	Rotate the Second Digit		
• Parts	32 50	71 84	23 54
• H(x)	82	55	77

Folding or Compression

➤ Example:

Divide a word into 5-bit substrings

$$h(\text{YET}) = 11001 \text{ XOR } 00101 \text{ XOR } 10100 = (01000)_2 = 8$$

$$h(\text{YOU}) = 11001 \text{ XOR } 01111 \text{ XOR } 10101 = (00011)_2 = 3$$

Discussion:

- Same value for a permutation of words ($h(\text{dad}) = h(\text{add})$)
- Possible solution: right circular shift (i bits for i-th substring)
- $h(\text{char}) = 10001 \text{ XOR } 01000 \text{ XOR } 01010 = 19$
 $h(\text{arc}) = 10000 \text{ XOR } 10100 \text{ XOR } 01100 = 8$

$c = 00011$
 $a = 00001$
 $r = 10010$

Medium square

- Mean-square function squares the key, extracts the R digits in the middle divided by the hash table size N.
- Ex: for a hash table with 100 entries and a key of 453, the square of the key is $453 * 453 = 205209$ and returns the middle two digits 52.

Extraction

➤ Extract and concatenate a few bits of the word

➤ Example:

$h(E)$ = bits number 1,2,7 and 8 of E

YET = 11001 00101,10100 $\Rightarrow h(YET) = (1000)_2 = 8$

YOU = 11001 01111,10101 $\Rightarrow h(YOU) = (1101)_2 = 13$

B = 00010 $\Rightarrow h(B) = (0010)_2 = 2$

Discussion:

➤ The choice of bits depends on the length of the words

➤ Does not provide a good result because the hash does not use all the information in the key.

Multiplication

- $H(\text{key}) = [m \times ((\text{key} \times \theta) \bmod 1)]$
 - θ is a value $0 < \theta < 1$

- Example

Let $\theta = 0.6125423371$, $m = 30$

$$h(\text{YET}) = (25780 * 0.6125423371) \bmod 1 * 30 = 10$$

Discussion:

- The size of the table is irrelevant
- Choose θ carefully:
 - Stay away from 0 and 1
 - $\theta = \frac{\sqrt{5}-1}{2} = 0.61803398874989484820458683436564$
 - Or $\theta = 1 - \frac{\sqrt{5}-1}{2} = 0.38196601125010515179541316563436$

Characteristics of a good hash function?

- Easy and quick to calculate
- Evenly distributes key values in the hash table
- Uses all information provided in the key
- **Minimizes collisions**

How to search for an element E ?

- $\text{Calculateh}(E) = \text{index}$
 - ❑ If $\text{Tab}[\text{index}]$ is empty then $E \notin \text{Tab}$
 - ❑ If $\text{Tab}[\text{index}] == E$ so $E \in \text{Tab}$
 - ❑ If $\text{Tab}[\text{index}] \neq E$ then search elsewhere (by direct chaining or open addressing)
-
- A search generally requires 1 to 5 hits

Collisions

- Collision is the condition in which 2 keys are stored in the same place.

Collision Resolution Techniques

- The process of finding another location is called collision resolution.
- There are a number of collision resolution techniques, and the most popular are open addressing and chaining.
- **Direct Chaining**: A linked list array
 - Separate chaining
 - Coalescing Hash
- **Open addressing**: Array-based implementation
 - Linear survey (linear search)
 - Quadratic survey (non-linear search)
 - Double hashing (using two hash functions)

Open Addressing

- In open addressing, all elements will be stored in the hash table itself.
- The approach is also known as **closed hash**.
- This procedure is based on the probing.
- A collision is resolved by probing.

Open Addressing

* Linear probing

- The interval/step between probes is set to 1.
- Search the hash table sequentially from the original hash location.
- If a slot is occupied, check the next slot.
- Wrap from the last table location to the first table location if necessary.
- The re-hash function: $rehash(key) = (n + i) \% table_size$ ($i=1,2..$)
- The step size should be relatively non-divisive of the table size (their common divisor should be 1).
- Problem: Table elements tend to cluster together in the hash table.

Open Addressing

* Quadratic probing

- The interval between probes increases proportionally to the hash value.
- The clustering problem is eliminated.
- Start from the original hash location i . If the slot is busy, check the slot for example $i+1^2$, $i+2^2$, $i+3^2$, $i+4^2$,...
- Wrap from the last table location to the first table location if necessary.
- The re-hash function: $rehash(key) = (n + k^2) \% table_size$

Open Addressing

* Quadratic probing

➤ The re-hash function: $rehash(key) = (n + k^2) \% table_size$

➤ Example:

Consider the hash table of size 11
 $h(key) = key \% 11$

➤ Insert keys:

$31 \bmod 11 = 9$

$19 \bmod 11 = 8$

$2 \bmod 11 = 2$

$13 \bmod 11 = 2 \rightarrow 2 + 1^2 \bmod 11 = 3$

$25 \bmod 11 = 3 \rightarrow 3 + 1^2 \bmod 11 = 4$

$24 \bmod 11 = 2 \rightarrow 2 + 1^2 \bmod 11 = 3 \rightarrow 2 + 2^2 \bmod 11 = 6$

$21 \bmod 11 = 10$

$9 \bmod 11 = 9 \rightarrow 9 + 1^2 \bmod 11 = 10 \rightarrow 9 + 2^2 \bmod 11 =$

$\rightarrow 9 + 3^2 \bmod 11 = 7$

	0
	1
2	2
13	3
25	4
5	5
24	6
9	7
19	8
31	9
21	10

Open Addressing

* Double Hash

- The interval between probes is calculated by another hash function.
- Double hashing reduces clustering better.
- The second hash function h_2 must be: $h_2(\text{key}) \neq 0$ and $h_2(\text{key}) \neq h_1(\text{key})$
- First probe the location $h_1(\text{key})$.
- If the slot is busy, probe the slot $h_1(\text{key}) + h_2(\text{key})$, $h_1(\text{key}) + 2 * h_2(\text{key}), \dots$

Open Addressing

* Double Hash

➤ Example:

Or a size 11 table

$$h1(\text{key}) = \text{key} \% 11$$

$$h2(\text{key}) = 7 - (\text{key} \% 7)$$

➤ Insert keys:

$$58 \bmod 11 = 3$$

$$14 \bmod 11 = 3 \rightarrow 3 + (7 - 0) \bmod 11 = 10$$

$$91 \bmod 11 = 3 \rightarrow 3 + (7 - 0) \bmod 11 = 10$$

$$\rightarrow 3 + 2 * (7 - 0) \bmod 11 = 17 \bmod 11 = 6$$

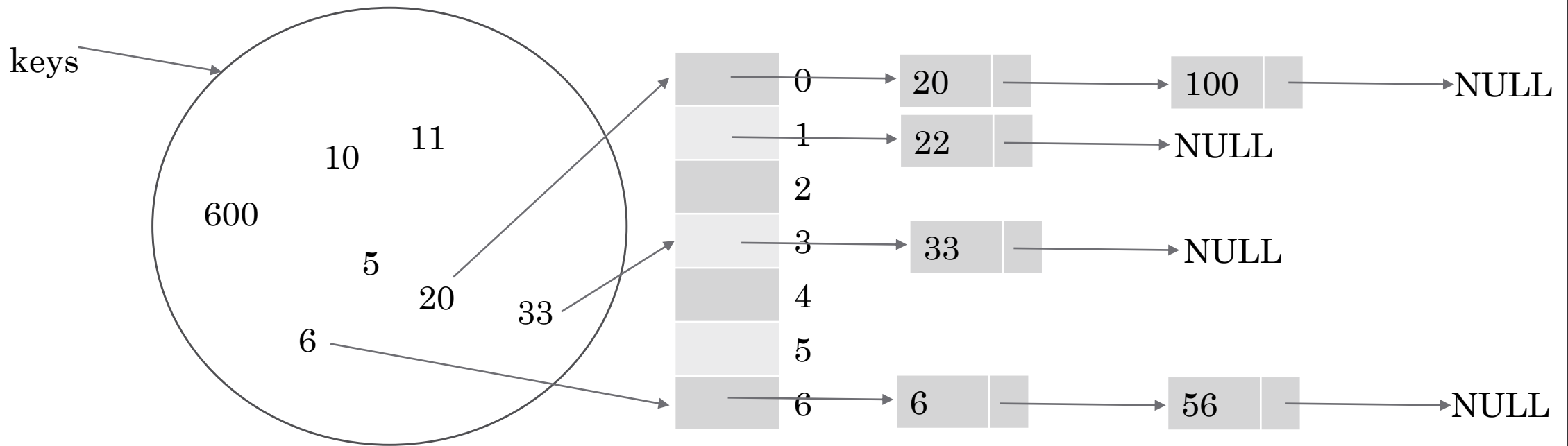
$$25 \bmod 11 = 3 \rightarrow 3 + (7 - 4) \bmod 11 = 6$$

$$\rightarrow 3 + 2 * (7 - 4) \bmod 11 = 9 \bmod 11 = 9$$

	0
	1
	2
58	3
	4
	5
91	6
	7
	8
25	9
14	10

Direct Chaining

* Separate chaining



- Collision resolution by chaining combines the representation of linked lists with the hash table.
- When 2 or more elements are hashed to the same location, these elements are stored in a linked list.

Direct Chaining

* Coalescent Hash

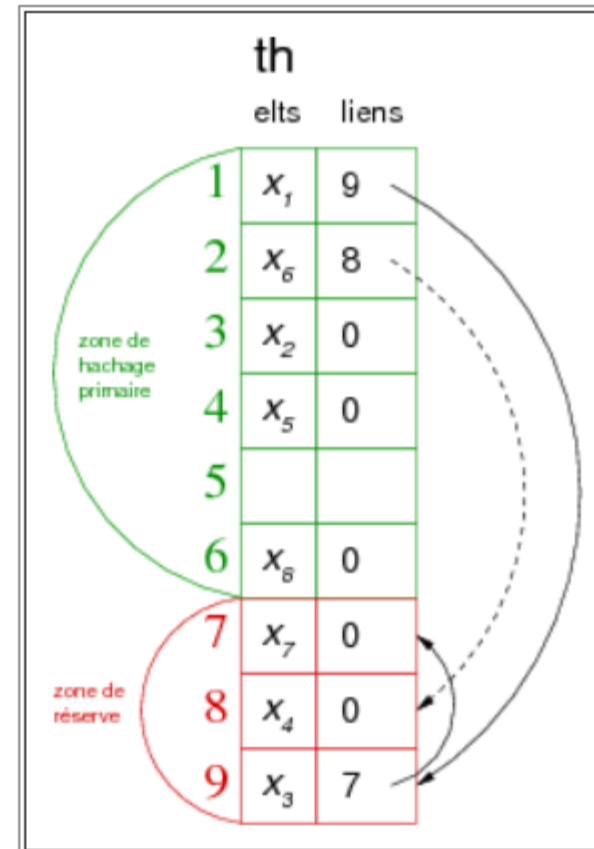
- If dynamic memory allocation is not possible → chain the elements together inside the hash array.
- Each entry in the array contains two fields: an element and a link (usually integer) representing the index of the array where the possible element in primary collision with it is located.
- The principle is to separate the hash table (of size m) in two zones:
 - A **primary hash zone** of p elements,
 - A **reserve area** of r elements to manage collisions.
- The constraints are to have $m = p + r$ greater than or equal to the number of elements.
- When an item is to be placed and its primary hash value reaches an empty square in the main area, it is placed there, otherwise, it is placed in the reserve area and the collision link is updated.
- Noticed: *The reserve area is always used in descending order of indices.*
- **Issue**: r small --> Reserve quickly filled, r large --> dispersion effect reduced

Direct Chaining

* Coalescent Hash

Avec $p=6$ et $r=3$, pour les valeurs d'exemple suivantes :

éléments	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
valeurs de hachage	1	3	1	3	4	2	1	6



Direct Chaining

* Coalescent Hash-LISCH

➤ LISCH = Late insert standard coalesced hashing

➤ Method:

1. $i = h(k)$;
2. Search from position i .
3. If k is not in the table, store k at the end of the table in position t pointing to the first empty cell from the end of the table. → k is at the end of the list of collisions

➤ Example:

$$h(k) = k \bmod 5$$

sequence: 1, 5, 21, 10, 15

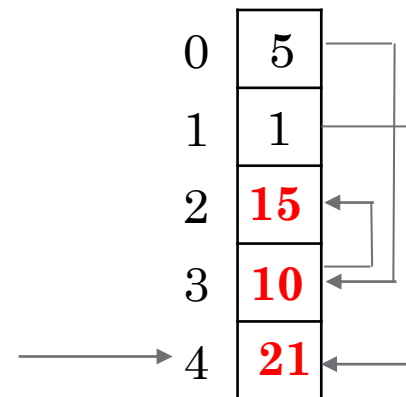


Table is full.

Direct Chaining

* Coalescent Hash-EISCH

➤ EISCH = Early insert standard coalesced hashing

➤ Method:

1. $i = h(k)$;
2. Search from position i .
3. If k is not in the table, store k at the end of the table in position t pointing to the first empty cell from the end of the table. → k is at the beginning of the list of collisions

➤ Example:

$$h(k) = k \bmod 5$$

sequence: 1, 5, 21, 10, 15

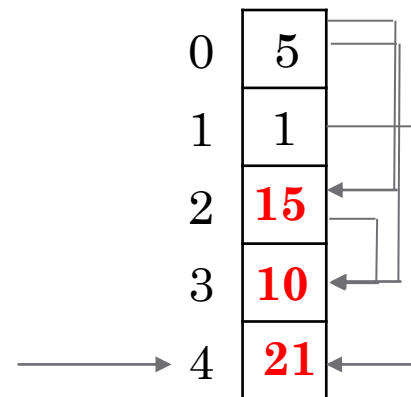
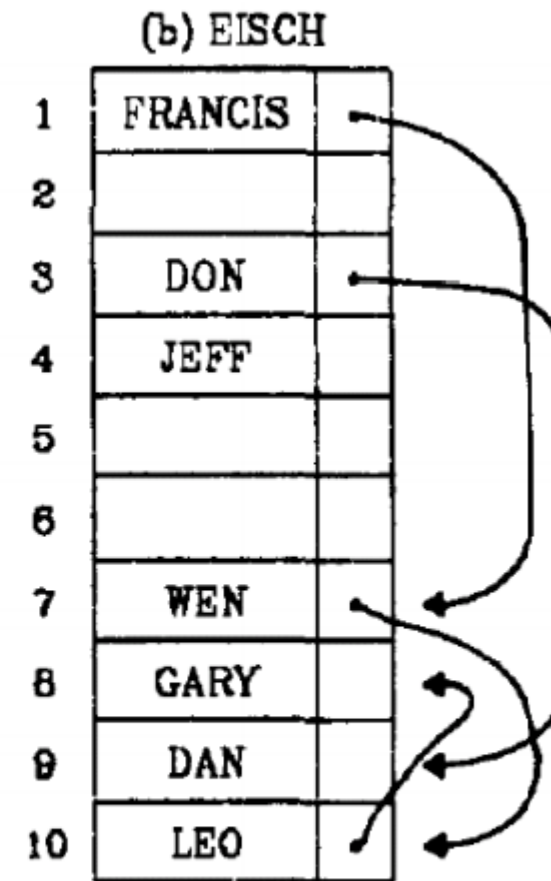
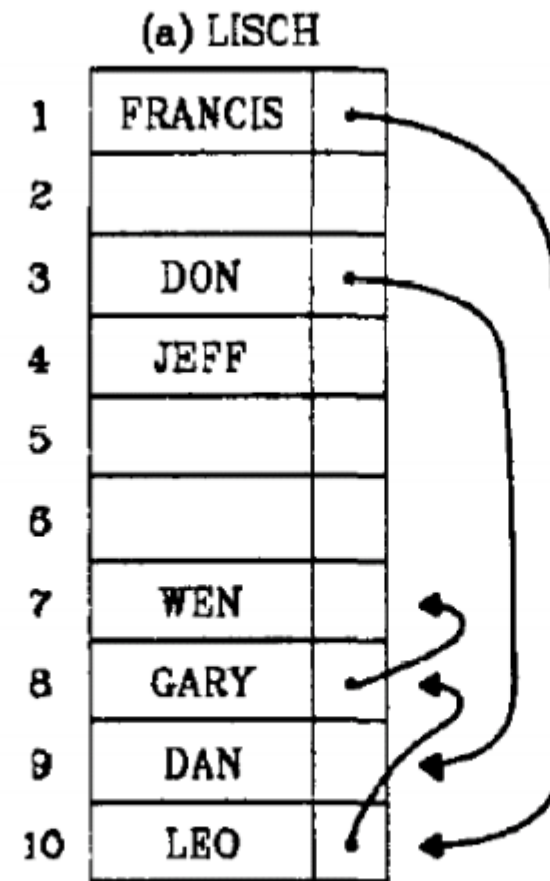


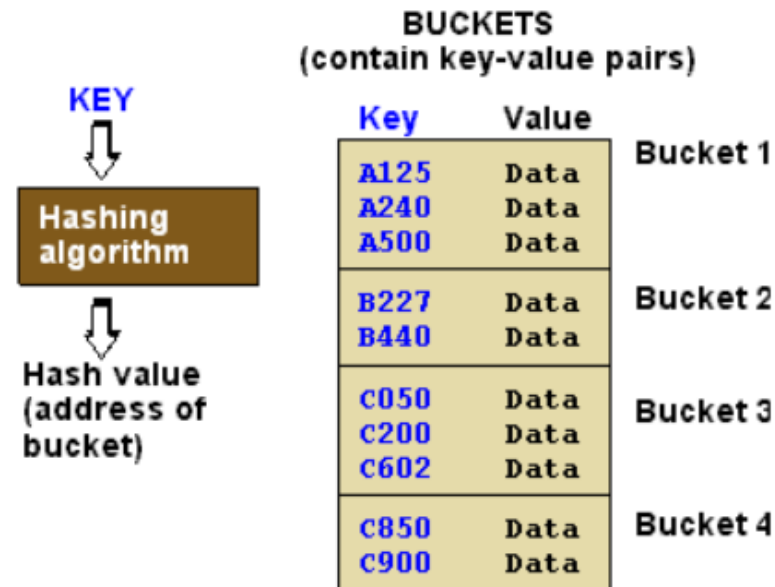
Table is full.



Keys: FRANCIS DON LEO JEFF DAN GARY WEN
 Hash Addresses: (a)(b) 1 3 1 4 3 10 1

Hashing by Buckets (Buckets)

- Divide the array into subarrays of the same size (buckets) and insert the keys into the buckets:
 - In a linear way
 - Using another hash function in each bucket.
 - ...
- Each bucket has its own places for collisions or one bucket of collisions is reserved for all buckets.





SUCCESS